

SCHOOLMS

Database Schema Documentation

86/100 Quality Score

Grade B Schema
Grade

22 Tables Generated

25 Rules Applied

Generated On	May 25, 2026 at 11:55
Domain	E Learning
Scale	Medium
GST Compliance	Yes
AI Provider	Groq — llama-3.3-70b-versatile

1. PROJECT OVERVIEW

A comprehensive school management system for student records and operations

Modules

1. Student Management	Manages student records and information
`unique_id_header_all`	stores unique identifiers for all records
`student_header_all`	stores student personal details
`batch_header_all`	stores batch information
`class_header_all`	stores class information
2. Fee Collection	Manages fee collection and payment tracking
`fee_transaction_all`	stores fee payment transactions
`payment_mode_configuration_all`	stores payment mode configurations
`fee_header_all`	stores fee structures and details
3. Attendance and Exam	Manages student attendance and exam records
`attendance_transaction_all`	stores student attendance records
`exam_header_all`	stores exam details and schedules
`quiz_header_all`	stores quiz details and schedules
4. GST Invoicing	Manages GST invoicing and compliance
`gst_invoice_header_all`	stores GST invoice details
`gst_configuration_all`	stores GST configuration and settings
5. Teacher Management	Manages teacher records and salary information
`teacher_header_all`	stores teacher personal details
`salary_transaction_all`	stores teacher salary payment transactions

Quality Validation

Score	86/100
Grade	B
Status	■ PASSED
Total Issues	7
Tables Found	22

2. TABLE-BY-TABLE DOCUMENTATION

For each table: purpose, column definitions, when to INSERT, when to UPDATE, and relationships.

1. `unique\_id\_header\_all`

Central registry for all business IDs. One row per entity type.

Column	Type	Notes
`id`	INT	
`table_name`	VARCHAR(100)	
`id_for`	VARCHAR(50)	
`prefix`	VARCHAR(20)	
`last_id`	VARCHAR(15)	
`created_on`	DATETIME	
`modified_on`	DATETIME	
■ INSERT when	Once per entity type during initial setup. Example: register 'student_header_all' with prefix 'STU-'.	
■ UPDATE when	Increment last_id and modified_on every time a new business ID is generated.	

2. `student\_header\_all`

Master entity table for Student records. One row per student entity.

Column	Type	Notes
`id`	INT	
`student_id`	VARCHAR(20)	
`name`	VARCHAR(100)	
`email`	VARCHAR(100)	
`phone_number`	VARCHAR(20)	
`address`	VARCHAR(200)	
`batch_id`	INT	
`class_id`	INT	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	
■ INSERT when	When a new student is registered or onboarded into the system.	
■ UPDATE when	When student profile information changes. Always copy old row to student_archive_all first.	

3. `batch\_header\_all`

Master entity table for Batch records. One row per batch entity.

Column	Type	Notes
`id`	INT	
`batch_id`	VARCHAR(20)	
`batch_name`	VARCHAR(100)	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	
■ INSERT when	When a new batch is registered or onboarded into the system.	
■ UPDATE when	When batch profile information changes. Always copy old row to batch_archive_all first.	

4. `class\_header\_all`

Master entity table for Class records. One row per class entity.

Column	Type	Notes
`id`	INT	
`class_id`	VARCHAR(20)	
`class_name`	VARCHAR(100)	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	
■ INSERT when	When a new class is registered or onboarded into the system.	
■ UPDATE when	When class profile information changes. Always copy old row to class_archive_all first.	

5. `fee\_transaction\_all`

Event/ledger table for fee operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	
`student_id`	INT	
`fee_id`	INT	
`payment_mode_id`	INT	

`payment_date`	DATE	
`amount`	DECIMAL(10, 2)	
`cgst_amount`	DECIMAL(10, 2)	
`sgst_amount`	DECIMAL(10, 2)	
`closing_balance`	DECIMAL(12, 2)	
`payment_method`	VARCHAR(50)	
`status`	TINYINT	1=paid, 2=unpaid
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

6. `payment\_mode\_configuration\_all`

Temporal configuration for payment_mode. Stores settings with effective date ranges.

Column	Type	Notes
`id`	INT	
`payment_mode_id`	VARCHAR(20)	
`payment_mode_name`	VARCHAR(100)	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	When a new configuration is set. Insert new row with new from_date. Do not update the old row — let it expire.
■ UPDATE when	Update status to 0 (past) when a newer configuration takes over.

7. `fee\_header\_all`

Master entity table for Fee records. One row per fee entity.

Column	Type	Notes
`id`	INT	
`fee_id`	VARCHAR(20)	
`fee_name`	VARCHAR(100)	
`fee_amount`	DECIMAL(10, 2)	

`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	When a new fee is registered or onboarded into the system.
■ UPDATE when	When fee profile information changes. Always copy old row to fee_archive_all first.

8. `attendance\_transaction\_all`

Event/ledger table for attendance operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	
`student_id`	INT	
`attendance_date`	DATE	
`attendance_status`	TINYINT	1=present, 2=absent
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

9. `exam\_header\_all`

Master entity table for Exam records. One row per exam entity.

Column	Type	Notes
`id`	INT	
`exam_id`	VARCHAR(20)	
`exam_name`	VARCHAR(100)	
`exam_date`	DATE	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	When a new exam is registered or onboarded into the system.
■ UPDATE when	When exam profile information changes. Always copy old row to exam_archive_all first.

10. `quiz\_header\_all`

Master entity table for Quiz records. One row per quiz entity.

Column	Type	Notes
`id`	INT	
`quiz_id`	VARCHAR(20)	
`quiz_name`	VARCHAR(100)	
`quiz_date`	DATE	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	
■ INSERT when	When a new quiz is registered or onboarded into the system.	
■ UPDATE when	When quiz profile information changes. Always copy old row to quiz_archive_all first.	

11. `gst\_invoice\_header\_all`

Master entity table for Gst Invoice records. One row per gst invoice entity.

Column	Type	Notes
`id`	INT	
`invoice_id`	VARCHAR(20)	
`invoice_date`	DATE	
`student_id`	INT	
`fee_id`	INT	
`amount`	DECIMAL(10, 2)	
`cgst_amount`	DECIMAL(10, 2)	
`sgst_amount`	DECIMAL(10, 2)	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	
■ INSERT when	When a new gst invoice is registered or onboarded into the system.	
■ UPDATE when	When gst invoice profile information changes. Always copy old row to gst_invoice_archive_all first.	

12. `gst\_configuration\_all`

Temporal configuration for gst. Stores settings with effective date ranges.

Column	Type	Notes
`id`	INT	
`gst_rate`	DECIMAL(5, 2)	
`cgst_rate`	DECIMAL(5, 2)	
`sgst_rate`	DECIMAL(5, 2)	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	When a new configuration is set. Insert new row with new from_date. Do not update the old row — let it expire.
■ UPDATE when	Update status to 0 (past) when a newer configuration takes over.

13. `teacher\_header\_all`

Master entity table for Teacher records. One row per teacher entity.

Column	Type	Notes
`id`	INT	
`teacher_id`	VARCHAR(20)	
`name`	VARCHAR(100)	
`email`	VARCHAR(100)	
`phone_number`	VARCHAR(20)	
`address`	VARCHAR(200)	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	When a new teacher is registered or onboarded into the system.
■ UPDATE when	When teacher profile information changes. Always copy old row to teacher_archive_all first.

14. `salary\_transaction\_all`

Event/ledger table for salary operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	
`teacher_id`	INT	
`salary_date`	DATE	

`amount`	DECIMAL(10, 2)	
`cgst_amount`	DECIMAL(10, 2)	
`sgst_amount`	DECIMAL(10, 2)	
`status`	TINYINT	1=paid, 2=unpaid
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ UPDATE when	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

15. `student\_archive\_all`

Historical mirror of student_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	
`student_id`	VARCHAR(20)	
`name`	VARCHAR(100)	
`email`	VARCHAR(100)	
`phone_number`	VARCHAR(20)	
`address`	VARCHAR(200)	
`batch_id`	INT	
`class_id`	INT	
`status`	TINYINT	1=active, 2=inactive
`created_on`	DATETIME	
`modified_on`	DATETIME	
`archived_on`	DATETIME	

■ INSERT when	Every time a row in student_header_all is about to be UPDATED. Copy the old row here first.
■ UPDATE when	Never update archive rows — they are immutable historical records.

16. `fee\_transaction\_archive\_all`

Historical mirror of fee_transaction_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	

`student_id`	INT	
`fee_id`	INT	
`payment_mode_id`	INT	
`payment_date`	DATE	
`amount`	DECIMAL(10, 2)	
`cgst_amount`	DECIMAL(10, 2)	
`sgst_amount`	DECIMAL(10, 2)	
`closing_balance`	DECIMAL(12, 2)	
`payment_method`	VARCHAR(50)	
`status`	TINYINT	1=paid, 2=unpaid
`created_on`	DATETIME	
`modified_on`	DATETIME	
`archived_on`	DATETIME	

■ INSERT when

Every time a row in fee_transaction_header_all is about to be UPDATED. Copy the old row here first.

■ UPDATE when

Never update archive rows — they are immutable historical records.

17. `attendance\_transaction\_archive\_all`

Historical mirror of attendance_transaction_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	
`student_id`	INT	
`attendance_date`	DATE	
`attendance_status`	TINYINT	1=present, 2=absent
`created_on`	DATETIME	
`modified_on`	DATETIME	
`archived_on`	DATETIME	

■ INSERT when

Every time a row in attendance_transaction_header_all is about to be UPDATED. Copy the old row here first.

■ UPDATE when

Never update archive rows — they are immutable historical records.

18. `salary\_transaction\_archive\_all`

Historical mirror of salary_transaction_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
--------	------	-------

`id`	INT	
`teacher_id`	INT	
`salary_date`	DATE	
`amount`	DECIMAL(10, 2)	
`cgst_amount`	DECIMAL(10, 2)	
`sgst_amount`	DECIMAL(10, 2)	
`status`	TINYINT	1=paid, 2=unpaid
`created_on`	DATETIME	
`modified_on`	DATETIME	
`archived_on`	DATETIME	

■ **INSERT when**

Every time a row in salary_transaction_header_all is about to be UPDATED. Copy the old row here first.

■ **UPDATE when**

Never update archive rows — they are immutable historical records.

19. `student\_life\_cycle\_all`

Status change audit trail for student_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	
`student_id`	INT	
`previous_status`	TINYINT	1=active, 2=inactive
`new_status`	TINYINT	1=active, 2=inactive
`status_change_date`	DATE	
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ **INSERT when**

Every time the status column in student_header_all changes. Record previous_status, new_status, and datetime.

■ **UPDATE when**

Never update life cycle rows — append only.

20. `fee\_transaction\_life\_cycle\_all`

Status change audit trail for fee_transaction_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	
`fee_transaction_id`	INT	
`previous_status`	TINYINT	1=paid, 2=unpaid

`new_status`	TINYINT	1=paid, 2=unpaid
`status_change_date`	DATE	
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	Every time the status column in fee_transaction_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

21. `attendance\_transaction\_life\_cycle\_all`

Status change audit trail for attendance_transaction_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	
`attendance_transaction_id`	INT	
`previous_status`	TINYINT	1=present, 2=absent
`new_status`	TINYINT	1=present, 2=absent
`status_change_date`	DATE	
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	Every time the status column in attendance_transaction_header_all changes. Record previous_status, new_status, and datetime.
■ UPDATE when	Never update life cycle rows — append only.

22. `salary\_transaction\_life\_cycle\_all`

Status change audit trail for salary_transaction_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	
`salary_transaction_id`	INT	
`previous_status`	TINYINT	1=paid, 2=unpaid
`new_status`	TINYINT	1=paid, 2=unpaid
`status_change_date`	DATE	
`created_on`	DATETIME	
`modified_on`	DATETIME	

■ INSERT when	Every time the status column in salary_transaction_header_all changes. Record previous_status, new_status, and datetime.
---------------	--

■ **UPDATE when**

Never update life cycle rows — append only.

3. PRODUCTION RULES APPLIED

The following 25 rules from the AI Database Rule Framework were applied during schema generation.

Rule ID	Rule Name	Category	Priority
7	Indian GST Compliance	taxation	CRITICAL
1	Naming Taxonomy	naming	CRITICAL
2	Two-ID System	identity	CRITICAL
9	Centralised ID Registry	identity	CRITICAL
54	Monthly Calendar Table Pattern	calendar	HIGH
51	Failed Transactions in Separate Table	financial	HIGH
98	Three-Table State Machine for Approvals	workflow	HIGH
11	Flat Approval Chains	workflow	HIGH
81	Generic Approval Engine	workflow	HIGH
45	Temp-to-Permanent Promotion	architecture	HIGH
6	Temporal Pricing System	financial	HIGH
27	Denormalised Running Balance	financial	HIGH
103	Multi-Tier Transaction Consolidation	financial	HIGH
3	Three-Layer Data Preservation	audit	HIGH
30	Timestamp Column Naming	naming	HIGH
4	Life Cycle Tracking	workflow	HIGH
86	Targets and Actuals in Separate Tables	financial	MEDIUM
95	Algorithm Parameters as Schema Columns	configuration	MEDIUM
83	Multi-Party Scheduling Inline	workflow	MEDIUM
40	Resume Pointer on Session Entities	architecture	MEDIUM
8	Status Integer Convention	convention	MEDIUM
74	Parallel Phase Statuses on Parent Entity	domain	MEDIUM
104	JSON Strings in Numbered Slot Columns	convention	MEDIUM
29	Decimal Money Fields	financial	CRITICAL
57	Pre-Computed Analytics at Write Time	financial	HIGH

4. DEVELOPER NOTES

Business ID Generation	All business IDs (student_id, fee_id, etc.) must be generated from unique_id_header_all. Insert a row to register each entity type with its prefix. Increment last_id on every new record. Never use the auto-increment 'id' column as a business identifier.
Foreign Key References	All foreign keys must reference the integer 'id' PRIMARY KEY column, not the business ID (varchar) column. The business ID is for display only. Join tables using integer id columns for performance.
Archive Table Usage	Copy the full row to the corresponding _archive_all table BEFORE making any UPDATE to a _header_all table. Archive tables store the history of every change. Never DELETE from _header_all — use status instead.
Life Cycle Table Usage	Insert a row into _life_cycle_all every time the status column changes. Store previous_status, new_status, and the datetime of the change. This enables full audit trail of every state transition.
GST Handling	Always store CGST and SGST amounts separately. Never combine them into a single tax column. Calculate at write time and store — never compute at read time. Current rate: CGST 9% + SGST 9% = 18% total (verify with your accountant for current applicable rates).
Closing Balance	The closing_balance column on transaction tables must be updated atomically with the transaction insert. Use a database transaction (BEGIN/COMMIT) to ensure the balance and the transaction record are always consistent. Never update balance separately.
Status Convention	Status 1 = active/success. Status 2 = inactive/failure. Status 3 = deleted/cancelled. Never hard DELETE rows. Set status = 2 or 3 and use soft delete everywhere.
Index Usage	All foreign key columns are indexed. Use the named indexes in your WHERE clauses. Avoid SELECT * in production — always select specific columns to use covering indexes effectively.